RAJESHGANESH
ROLL NO 241501161

EXPERIMENT 01

Low code versus traditional development

Aim

Compare low code and traditional application development for the same library requirements, including delivery effort, flexibility, security, operations, and long-term maintenance.

Procedure

1. Define a common scope: title and copy catalogues, member registration, issue and return, overdue reporting, permissions, and a CRM integration. Use the same 14-day loan term and three-open-loan limit for both approaches.

2. Sketch two architectures. Low code: Creator forms, managed storage, reports, pages, Deluge, and connections. Traditional: a web interface, application API, relational database, authentication service, deployment pipeline, and monitoring.

3. Trace one issue transaction through both architectures. Identify where required fields, member status, open-loan limit, copy allocation, authorization, and audit logging are enforced.

4. Break work into data model, interface, business rules, testing, integration, and deployment. If prototypes are built, time equivalent tasks and record actual effort. Otherwise label estimates and leave measured values blank.

5. Score each approach from 1 to 5 against agreed criteria and explain each score. Use weights chosen before scoring; do not present the resulting preference as universal.

Implementation and verification

Criterion	Creator approach	Traditional approach
Initial delivery	Reusable forms and reports reduce routine setup.	UI, API and database must be assembled.
Customization	Deluge, widgets and APIs extend the platform within supported boundaries.	Broad control over libraries, data structures and runtime.
Transactions	Verify platform guarantees for cross-record operations.	Database transactions and constraints can be designed directly.
Operations	Managed runtime; account limits and vendor changes matter.	Team operates deployment, backups and runtime upgrades.
Portability	Export data; plan rewrites of proprietary logic.	Portability depends on chosen services and architecture.
Cost	Evaluate users, portals, integrations and capacity.	Evaluate development, hosting and ongoing support.

Evidence and testing

E01: architecture sketches, a requirements checklist, justified scoring sheet, and either recorded timings or clearly labelled estimates. Check both options against the same acceptance tests, particularly double issue and restricted member access.

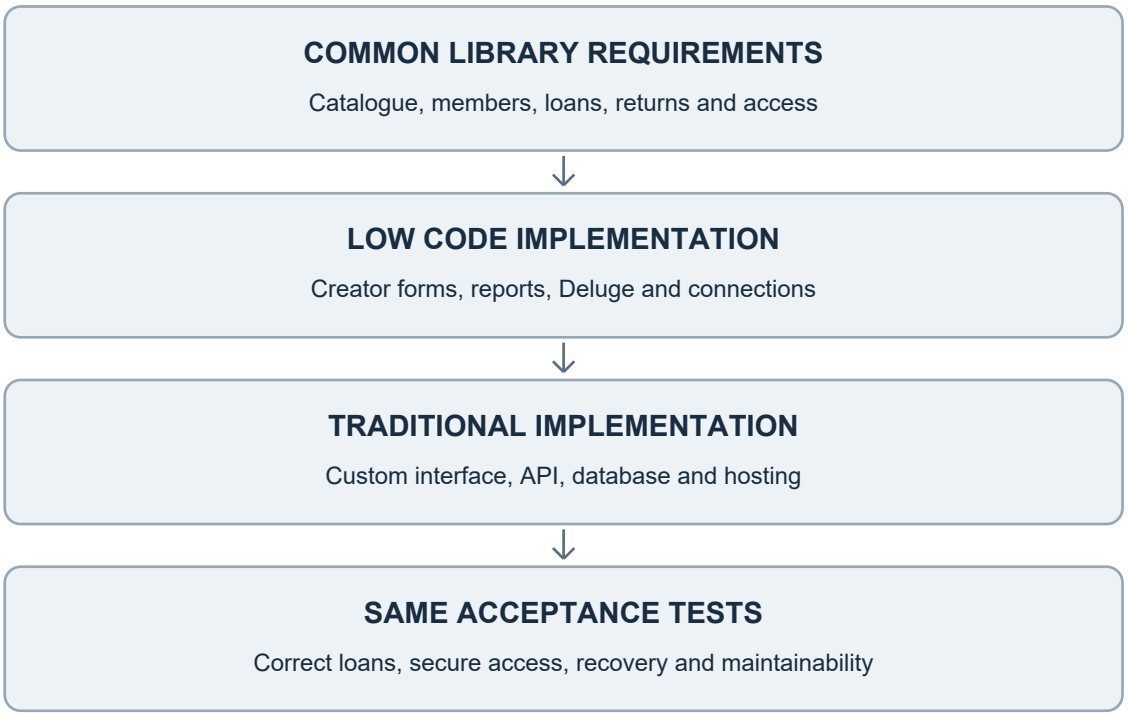
Implementation and interpretation

Compare equivalent scope rather than the visual appearance of the two implementations. A fast form prototype does not establish secure record access or safe concurrent issue processing. Include maintenance, training and the effort needed to recover a failed transaction in the comparison.

EXPERIMENT 01 VISUAL RECORD

Application images and explanation

Illustrative process diagram



Generated explanatory diagram. It is not a screenshot of an executed workflow.

Compare equivalent scope rather than the visual appearance of the two implementations. A fast form prototype does not establish secure record access or safe concurrent issue processing. Include maintenance, training and the effort needed to recover a failed transaction in the comparison.

Result

Expected result and verification criteria. The comparison identifies where Creator is suitable for a routine library workflow and where custom services may be justified. All cost and time claims have an explicit source, measurement, or estimate label.